

Efficacy of Proposed Strategies for PokerBots

- **Volatility/Risk-Sensitive Play:** In theory one could adjust the objective toward higher variance (“volatility-maximisation”), but game-theoretic analysis warns this is problematic. In fact, using risk-sensitive utilities (e.g. $\mu - \gamma\sigma^2$ or Sharpe objectives) yields no guaranteed Nash equilibrium and forces pure (deterministic) strategies ¹. Such pure strategies are **highly exploitable** in poker, so there is no evidence they improve long-term EV. In practice, top poker bots maximize expected value and then exploit leaks, rather than inflate variance.
- **Measuring Risk Impact:** We can track EV vs outcome variance using finance-style metrics. For example, compute the **Sharpe ratio** (mean return divided by std deviation) or Conditional Value-at-Risk of our bot’s returns ². By comparing Sharpe (or variance) between the baseline and modified agent (over many simulated matches), we can isolate the volatility effect. Crucially we would compare EV (mean payoff) and variance side-by-side to ensure EV is not harmed. In other words, report both mean EV and variability (or risk-adjusted EV) for each strategy change.
- **Copula-Based Activations:** Copula activations are an exotic way to couple features to model nonlinear dependencies. Recent work in survival analysis shows that **learnable copula-based activation functions** (e.g. Clayton or Gumbel copulas) can capture complex multivariate relationships and improve predictive accuracy ³. However, those gains appeared in very large CNN-LSTM models on high-dimensional biomedical data – not on tiny, fast decision systems. In a small, CPU-bound pokerbot, copula computations (which involve exponentials and solving for joint CDFs) likely add significant overhead. There is **no known benchmark** in low-latency games where copula activations beat simpler methods. In practice, standard MLP or attention layers (with concatenation or simple bilinear interactions) achieve similar coupling with far less cost. A copula-based layer might slightly improve accuracy if there were strong feature correlations to exploit, but the literature suggests the improvement is modest and domain-specific ³. For a small real-time bot, simpler feature couplings (e.g. learned linear layers on precomputed feature pairs) are much more cache-friendly and likely sufficient.
- **Graph-Isomorphic/GNN Activations:** Using a graph representation of the game state is promising. Recent work on **Graph Neural Networks (GNNs)** for games shows large performance gains when the game naturally forms a graph. For example, a chess bot (AlphaGateau) replaced CNN layers with a graph-attention network and **outperformed a CNN baseline with similar model size** ⁴. Similarly, GNNs enabled 10× faster learning in Othello/Gomoku (using GIN layers) compared to vanilla AlphaZero ⁵. This suggests that for poker, encoding the state (hands, bets, position) as a small graph and processing it with a graph-invariant network *might* improve strategic depth without extra parameters. However, GNN layers do incur additional matrix operations and may be slower per-action than a tiny MLP. On a single-core CPU, a small GNN with carefully optimized sparse operations might just meet the 60-second per-1000-hands limit, but a large GNN would not. Overall, graph-based feature interactions **could outperform standard fully-connected layers** on complex relational features (position, card relationships), as shown in other games ⁴, but the benefit must be weighed against higher compute cost.
- **Continuous Random-Draw vs Stochastic Policies:** Injecting randomness in the network (a “continuous random-draw” activation) is not a standard approach. In poker RL, exploration is normally handled by **explicit stochastic policies**: the network outputs a probability distribution over actions, and we sample from it (or add entropy bonuses) ⁶. Both methods achieve similar ends

(randomization of play). There is no known evidence that randomizing internal activations yields any advantage over simply sampling actions. In fact, noisy activations could destabilize learning and violate determinism. Thus we expect **no clear EV benefit** from that novelty; conventional mixed strategies (softmax policies or ϵ -greedy) suffice for exploration.

- **Hebbian/Mimetic Online Adaptation:** Hebbian (correlation-based) or mimetic (copying) learning rules offer a form of very fast adaptation using only local updates. In neural terms, a reward-modulated Hebbian rule (weight change $\propto \text{pre} \times \text{post} \times \text{global reward}$) has been shown to **optimize a network under noisy conditions** ⁷. In other words, such rules *can* tune weights during play (as a form of “test-time adaptation”). However, in an adversarial game this raises risks: if the bot adapts its strategy on every match, opponents can exploit the change (non-stationarity). Empirically in poker AI research, explicitly adapting mid-match to a single opponent tends to cause overfitting and circular adaptation. A lesson from computer poker tournaments is that a strategy that tries to exploit one opponent may perform worse on unseen opponents ⁸. In practice, any Hebbian rule would need strong safeguards (e.g. very slow learning rate, weight clipping) to avoid drifting far from a robust baseline. Without careful design, online adaptation would likely **destabilize play and increase exploitability** despite its local convergence proof ⁷.

Failure Modes & Mitigations: All these approaches risk common pitfalls. **Overfitting** is key: e.g. heavy adaptation or complex models may fit particular opponents well but fail in general ⁸. Mitigate via extensive self-play or opponent pools, and by testing on hold-out bots (Johanson’s experiments showed “holdout” opponents crushed over-tuned strategies ⁸). **Exploitability** is another danger: pure or non-mixed strategies can be taken advantage of. In fact, in fixed tournaments the best outcome is total chips won, and a high-variance (“maniacs”) strategy can beat a cautious equilibrium by winning big sometimes ⁹. To avoid such swings, ensure any new method remains near Nash equilibrium (e.g. limit deviations) or add smoothing. **Non-stationarity** occurs if the bot changes behavior too quickly (e.g. from Hebbian updates), which opponents can detect. Solution: limit online learning to long horizons, or freeze policy for whole matches. Generally, each approach must be validated with duplicated-seat tests (to cancel luck) and rollback capability in case EV degrades.

Compatibility & Constraints: We need 100% CPU, deterministic code, and cache-friendly models. Approaches that require Monte Carlo or GPU (e.g. heavy sampling, large CNNs) are out. Among the above, simple linear or GNN layers (with fixed graph structure) can be made deterministic and cached. Copula layers (if novel) are likely slow and require transcendental functions – they risk hitting per-action time limits. Graph networks and Hebbian rules (if implemented with fixed update rules) are deterministic, but their overhead is significant. The *most* compatible strategies are those built from fixed linear algebra (vector ops) and no random seeds in inference – for example a small GNN or a static utility adjustment. Those 2–3 that fit best might be: (a) a tiny graph-based network (if it fits in time), (b) a limited adaptive rule (extremely slow Hebbian tweaks), and (c) traditional regression features with a risk-term adjustment (like a small dynamic threshold) which doesn’t require randomness. Any other novelty likely violates the single-core, deterministic constraint or adds too much CPU load.

Operational & Security Considerations: There are no new data-privacy issues here (no personal data used), but adding complex models increases attack surface: we must ensure no hidden backdoors or data leakage. From an engineering viewpoint, each new strategy should be behind a feature flag (allowing gradual rollout and rollback). **Observability:** instrument the bot to log EV, variance, hand distributions, and any internal metrics (e.g. Hebbian weight drift) so failures can be spotted. **Performance:** measure per-decision latency; profile cache use. If a new layer slows response above budget, it must be tuned or removed. **Versioning:** any learned parameters or model weights should be versioned carefully; updates to

feature distributions (e.g. card encodings) must match model training to avoid mismatch. Keep a clear roll-back plan: if evaluation results (or live metrics) degrade, restore the previous agent immediately.

Recommended Approaches: Based on the above, the *most promising* strategies to prototype are:

1. **Graph Neural Network layers** for relational features. This has demonstrated success in other strategic games ⁴ ⁵ and could capture subtle interactions (e.g. pot size vs hand strength across positions). A small GNN can be made deterministic and may boost EV by better generalization.
2. **Selective nonlinear feature couplings** (in lieu of full copula). Rather than a full copula transform, one could hand-design or learn specific feature interactions (e.g. pairwise products or decision trees) that approximate important dependencies. This keeps runtime low while still capturing extra structure. If copula functions are tried, only very simple ones (like a fixed small joint CDF) should be used given latency constraints, and tested against a baseline.
3. **Conservative online adaptation (Hebbian-ish)** with strict limits. For example, allow a tiny learning rate update to the network only once every N hands (with smoothing). This lets the bot adapt slowly to an opponent without inducing large nonstationarity. The rule can be Hebbian ($\Delta w \propto \text{reward} \times \text{local activation}$) but with capping. If even this is risky, simply using opponent modeling outside the core policy may suffice (i.e. not modify core net at all during play).

Each of these should be implemented cautiously (e.g. behind flags, with duplicate-match testing). Other ideas (pure variance-maximising objectives, complex copula nets, random-activation exploration) look less viable given constraints and lack of evidence.

Sources: We drew on game-theoretic and ML research. For instance, Horáček’s analysis shows risk-sensitive payoffs break Nash conditions ¹. Sharpe/mean-variance metrics are standard for risk-adjusted return ². Recent work in *structural* nets (copula or graph) indicates potential gains in complex domains ³ ⁴, though at computational cost. Results from computer-poker tournaments highlight exploitability and evaluation procedures (duplicate-seat matches) ¹⁰ ⁸ ⁹. Finally, biologically-inspired learning rules have been shown to work *in principle* under noise ⁷, but require care in practice.

Options Comparison

Approach	Description	EV Impact (theory)	Compute/Latency	Determinism	Risk/Exploitability
Volatility-Maximization (Sharpe/CVaR)	Modify objective to favor high-variance outcomes ¹ .	No proven EV gain. Theory predicts either no equilibrium or pure strategies (bad).	Low overhead (just objective shift).	Deterministic, but mixing needed to avoid pure exploitability.	High: leads to pure (exploit-prone) play ¹ .

Approach	Description	EV Impact (theory)	Compute/Latency	Determinism	Risk/Exploitability
Copula-based Features	Use copula functions in activations to capture nonlinear feature dependence ³ .	Potential small accuracy gain in rich correlated data.	High: expensive math per action, hard on CPU.	Deterministic (fixed functions) but heavy compute.	Moderate: extra complexity can overfit on limited data.
Graph Neural Network	Encode state as graph; use GNN/ GAT layers (e.g. AlphaGateau) ⁴ .	Empirically strong in other games; could increase EV by better generalization.	Medium-High: extra matrix ops; small GNN might fit in 60s, large GNN may not.	Deterministic (weights fixed), depends on random init only at training.	Low to moderate: structured, but any model can be overfitted.
Continuous Random Activation	Add random noise to activations each decision.	<i>No known benefit.</i> Equivalent to a stochastic policy but less interpretable.	Low (just noise injection), but likely no EV upside.	Non-deterministic by design (not allowed per specs).	High: randomness breaks determinism; may destabilize policy.
Hebbian/Mimetic Learning	Online weight updates from local rule (reward×pre×post) ⁷ .	Can adapt to opponent quickly in theory; potential EV gain vs very static bots.	Low per update (simple multiplications), updates maybe every few hands.	Non-deterministic (learning rate introduces change), but can be made nearly-deterministic by fix seed.	High: rapid adaptation can be exploited, causes non-stationarity. Must severely constrain updates.

Recommended Approach

Given the single-core, deterministic constraints and current evidence, we recommend prioritizing **graph-based feature networks** and **selective feature couplings**, with cautious adaptation rules. A compact GNN (or graph attention) could allow richer interactions (piecewise skill combinations) without random sampling, and has been shown to *outperform* standard nets in strategic games ⁴. Copula functions seem overkill; instead, we suggest implementing a few key non-linear feature combinations (e.g. pairwise or learned embedding features) that approximate the same idea but at trivial cost. Online learning rules (Hebbian) could be tested in limited form (very low learning rate, perhaps only between matches), but only after ensuring a fallback to the original policy to avoid runaway losses.

The choice is justified by balancing theoretical gain vs practical cost: the literature shows graph nets and structured features can capture domain symmetries and have tangible performance gains ⁴ ³ . In contrast, trying to *maximize volatility* or add heavy randomness is either provably detrimental ¹ or lacks support. We will proceed incrementally: first integrate and benchmark a small GNN layer, validate it (duplicate-seat tests against baselines), then consider any small feature engineering improvements. If those succeed within time budget, we can cautiously test very slow Hebbian tweaks; if not, we will focus on the proven elements (GNN + fixed couplings).

Implementation Considerations

- **Testing & Evaluation:** Use a *self-play harness* and a diverse opponent pool for robust evaluation. As in past competitions, run **duplicate matches** (each scenario played from both seats) and rank by total winnings ¹⁰ . Include holdout bots to catch overfitting (Johanson's results showed overfit models fail on unseen opponents ⁸). Perform seat swaps (switch dealer button) to eliminate positional bias.
- **Determinism & Rollback:** Implement new strategies behind feature flags. Ensure the bot's decision logic has a fixed RNG seed or none at inference time. Benchmark per-decision latency; if the new layer exceeds the 60s budget, disable it. Maintain a clear rollback path: if EV drops or variance spikes, revert to the previous version immediately.
- **Observability:** Instrument logging of key metrics – EV per 1000 hands, win/lose variance, hand distributions, and any new-module outputs. For the learning rule (if used), log weight changes. Correlate observability data with feature flags to quickly identify what change caused an issue.
- **Security/Privacy:** No direct PII or external data is used, so privacy risk is low. However, complex code can introduce vulnerabilities. Use static analysis and code reviews for any new math or update routines. Keep models and rules versioned and check-summed to prevent tampering.
- **Compatibility:** Keep all new code Python 3.7-compatible (per the environment) and rely only on standard libraries or approved dependencies. Avoid GPU or multi-threading – profiles must run on the existing single-core setup. Document software versioning (e.g. NumPy, any ML libs) so the setup can be reproduced exactly.
- **Operational:** Monitor bot performance during matches. If a strategy change causes a decline or bug, the experiment must be stopped and configuration rolled back. Use progressive deployment (e.g. test on held-out simulated games before any live use).

Bibliography / Links

- Horáček, M. "Risk-Aversion in Algorithms for Poker" (MSc thesis, Masaryk University 2016) ¹ ² .
- Kim, J.-M. *et al.* "Deep learning-based survival analysis with copula-based activation functions" (Computational Statistics 2025) ³ .
- Rigaux, T. & Kashima, H. "Enhancing Chess Reinforcement Learning with Graph Representation" (arXiv 2024) ⁴ .
- Johanson, M. "Robust Strategies and Counter-Strategies: Building a Champion-Level Computer Poker Player" (MSc thesis, Univ. of Alberta 2008) ⁸ ¹⁰ ⁹ .
- Legenstein, R. *et al.* "A Reward-Modulated Hebbian Learning Rule Can Explain...Network Reorganization" (J. Neurosci. 2010) ⁷ .

1 2 Risk-Aversion in Algorithms for Poker

<https://is.muni.cz/th/ydbvx/thesis.pdf>

3 (PDF) Deep learning-based survival analysis with copula-based activation functions for multivariate response prediction

https://www.researchgate.net/publication/395536159_Deep_learning-based_survival_analysis_with_copula-based_activation_functions_for_multivariate_response_prediction

4 5 Enhancing Chess Reinforcement Learning with Graph Representation

<https://arxiv.org/html/2410.23753v1>

6 Deterministic vs. Stochastic Policies in Reinforcement Learning | Baeldung on Computer Science

<https://www.baeldung.com/cs/rl-deterministic-vs-stochastic-policies>

7 A Reward-Modulated Hebbian Learning Rule Can Explain Experimentally Observed Network Reorganization in a Brain Control Task - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC2917246/>

8 9 10 poker.cs.ualberta.ca

<https://poker.cs.ualberta.ca/publications/johanson.msc.pdf>